MARCO 1 · IDEAÇÃO & DESENVOLVIMENTO INICIAL

SmartStock Vision — da primeira ideia ao protótipo inicial

Como uma proposta de visão computacional para perdas de varejo virou, depois de duas rodadas de revisão, o sistema de cesta virtual que já funciona hoje.

DISCIPLINA / TURMA
Laboratório de Inteligência Artificial — prof. Diego

GRUPO
Judith Otávia Margarido de Andrade

DATA
21 set. 2026

REPOSITÓRIO
`smartstock-vision`

Nota de método (mesma convenção usada no documento de investigação técnica do grupo): **fonte confirmada** = verificado em documentação oficial, artigo científico ou no próprio código do repositório, citado com link ou caminho de arquivo. **Sugestão da IA** = análise ou dedução, não extraída de uma fonte específica. **A confirmar com o professor** = decisão que depende de uma discussão que a IA não presenciou diretamente.

01 A ideia original / 02 Diagnóstico / 03 Pesquisa / 04 Proposta revisada / 05 Segundo pivot / 06 O que já foi construído / 07 Banco de dados / 08 Arquitetura atual / 09 Privacidade & LGPD / 10 Próximos passos / 11 Uso de IA / 12 Referências

> O projeto não nasceu como o sistema de hoje — passou por **duas rodadas de revisão de escopo** antes de chegar ao protótipo atual. A primeira (documentada na investigação técnica do grupo) reduziu uma ideia ampla demais de visão computacional a um MVP focado. A segunda — visível hoje só no código, não em um documento formal — foi mais radical: em vez de depender de um modelo de visão treinado desde o início, o time separou *"como o produto é detectado"* de *"o que o sistema faz com essa detecção"*, criando três fontes de detecção intercambiáveis (`FonteDeteccao.SIMULADO` , `SENSOR_PESO` , `CAMERA_REAL`). Isso permitiu entregar um sistema de cesta virtual, estoque e conferência de pagamento genuinamente funcional *antes* mesmo de haver um único modelo de IA treinado — deixando a visão computacional real para o Marco 3, já especificado em documento próprio.

01 A ideia original

O grupo não guardou o texto exato da proposta inicial como um documento à parte — mas seu conteúdo pode ser reconstruído com segurança a partir do que o documento de investigação técnica do grupo

07 Banco de dados

O modelo de dados (fonte confirmada — `backend/app/models/models.py`) é gerenciado como ORM

diagnosticou sobre ela (Etapa 1). A proposta original combinava, num único sistema, três problemas de visão computacional distintos:

- **Inventário/contagem** — quantos produtos existem numa prateleira, a partir de imagem.
- **Rastreamento de produtos retirados** — seguir um item específico desde a prateleira até a saída.
- **Conferência de pagamento (anti-furto)** — cruzar o que foi detectado com o que foi efetivamente pago.

Motivação de negócio (fonte confirmada — BizTech Magazine, 2025): perdas de varejo ("shrink") somaram uma estimativa de **US$ 132 bilhões globalmente em 2024**, contra US$ 112 bilhões em 2022 — um problema real e crescente, o que validava a escolha do domínio mesmo antes de qualquer ajuste de escopo.

02 Diagnóstico da proposta original

Antes de escrever uma linha de código, o grupo submeteu a própria ideia a uma investigação técnica crítica. Quatro problemas foram identificados (fonte confirmada — documento de investigação, Etapa 1):

1. **Escopo amplo demais para um semestre** — os três problemas acima são, cada um, uma linha de pesquisa ativa isoladamente; tentar os três ao mesmo tempo era risco alto de não terminar nenhum bem.
2. **Ambiguidade entre "inventário" e "prevenção de perdas"** — são tarefas de IA diferentes: a primeira é principalmente detecção + contagem; a segunda envolve rastreamento de objetos *e pessoas* ao longo do tempo, uma tarefa mais sensível.
3. **A tarefa de IA não estava definida em termos técnicos precisos** — classificação? detecção? rastreamento? cada resposta implica um pipeline diferente.
4. **Risco de privacidade subestimado** — câmeras + produtos retirados quase inevitavelmente capturam imagens de pessoas, o que é dado pessoal (e, se envolver reconhecimento facial, dado biométrico sensível) pela LGPD.

Conclusão do diagnóstico: a ideia era viável, mas precisava de redução de escopo formal — o que motivou a Etapa 3 do documento de investigação (ver Seção 04 abaixo).

03 Pesquisa que embasou a decisão

Antes de propor um MVP, o grupo levantou datasets públicos e técnicas já usadas na literatura para esse domínio específico (fonte confirmada, com link/citação no documento de investigação, Etapa 2):

DATASET	FINALIDADE

SQLAlchemy, com migrações versionadas via **Alembic** (`backend/alembic/`), rodando em **SQLite** no ambiente de desenvolvimento do grupo e preparado para **PostgreSQL** em produção — a mesma tabela, sem reescrever nada, só troca a `DATABASE_URL` . São seis tabelas, já em uso pelo protótipo:

TABELA	CAMPOS PRINCIPAIS
`usuarios`	nome, email, senha_hash, pape (`ADMIN` / `PROPRIETARIO` / `OPE`
`produtos`	nome, preço, quantidade_estoc estoque_mínimo, peso_unitári
`sessoes_compra`	código (`SS-XXXXXX` , único), st valor_total, pago_em
`itens_cesta`	sessao_id + produto_id (únicos quantidade_detectada, quantic
`prateleiras`	nome, produto_id, peso_atual_

SKU-110K	Detecção em prateleiras densamen compactadas (classe única: "produto")
RPC (Retail Product Checkout)	Cenas de checkout com múltiplos produtos — o mais próximo da idei original
Freiburg Groceries	Classificação de mercearia em foto do mundo real
RP2K	Classificação fina entre produtos muito parecidos

Técnicas comparadas: **YOLOv8** foi preferido a **YOLO26** (mais novo, lançado em jan/2026, mas com menos comunidade e tutoriais) para reduzir o risco de a equipe travar sem solução documentada; **ByteTrack** foi preferido a **DeepSORT** para rastreamento, por ser mais leve computacionalmente. Um trabalho relacionado real serviu de calibrador de expectativas: Shoman et al. (2022) obtiveram **F1-score de 0,44** num problema semelhante de contagem automática para checkout — evidência de que é um problema difícil, longe de "resolvido" na literatura.

04 Proposta revisada (primeira ideia formalizada)

Com base no diagnóstico e na pesquisa, a Etapa 6 do documento de investigação formalizou um MVP de escopo reduzido — esta é, na prática, a **primeira ideia concreta e executável** do projeto:

DENTRO DO MVP ORIGINAL

- 3 a 5 produtos cadastrados (não um catálogo inteiro)
- Vídeo gravado, não câmera ao vivo
- YOLOv8 com fine-tuning detectando esses produtos
- Regra de "zona": produto saiu de

FORA DO MVP ORIGINAL

- Câmera em tempo real
- Catálogo com dezenas/centenas de produtos
- Rastreamento de pessoas / comportamento suspeito
- Integração com pagamento real
- Aplicativo mobile

`eventos_deteccao`	tipo (retirada/devolução), quan (`SIMULADO` / `SENSOR_PESO` / `C` data_hora

As relações seguem uma cadeia simples: uma `SessaoCompra` tem vários `ItemCesta` (um por produto na cesta) e vários `EventoDeteccao` (um por retirada/devolução registrada); cada `Prateleira` aponta para um único `Produto`. É esse desenho — separar "o que está na cesta agora" (`ItemCesta`, um valor atual) de "o que aconteceu" (`EventoDeteccao`, um histórico que só cresce) — que permite ao sistema reconstituir qualquer divergência depois, sem depender de recalcular nada a partir de logs externos.

08 Arquitetura atual

CAMADA	TECNOLOGIA
Backend / API	Python 3 — FastAPI, SQLAlchemy
Banco de dados	PostgreSQL (produção) / SQLite (desenvolvimento)
Painel / frontend	TypeScript, React, Vite, Tailwind CSS
Detecção — simulada	Clique manual no painel
Detecção — sensor de peso	Delta de peso → unidades
Detecção — câmera real	YOLOv8 fine-tuned (Etapa 03)

O documento de especificação técnica do Marco 3 (já existente, ver Referências) detalha exatamente como a terceira fonte — `CAMERA_REAL` — vai se encaixar neste mesmo pipeline, sem alterar as camadas já

uma area definida?

- Registro em SQLite + dashboard em Streamlit

Essa proposta já vinha com métricas de sucesso definidas (F1-score e mAP para qualidade de detecção, FPS para viabilidade prática) e um cronograma de 16 semanas — mas dependia, desde a semana 3, de um modelo de visão computacional treinado para o sistema fazer qualquer coisa útil.

05 O segundo pivot: da visão computacional pura à cesta virtual

Esta decisão não está registrada num documento formal — mas está registrada, de forma verificável, nos comentários e na estrutura do próprio código-fonte (fonte confirmada, ver caminhos de arquivo abaixo). Em vez de construir o sistema *em torno* de um modelo de visão computacional, o grupo isolou a detecção como uma interface plugável:

ASPECTO	PROPOSTA REVISADA (SEÇÃO 04)	CONSTRUÍDO AGO
Fonte da detecção	Só câmera + YOLOv8, desde o início	Enum `FonteDet` `SIMULADO` , `SEN` `CAMERA_REAL`
O que já funciona de verdade hoje	Nada, sem o modelo treinado	Clique simulado e — ambos já conve reais na cesta virt
Unidade central do domínio	Evento de "produto saiu da zona"	`SessaoCompra` conferência de pa reconciliação de e
Stack	Streamlit + SQLite	FastAPI + React/V SQLAlchemy (Pos

Por que isso importa (sugestão da IA): o MVP original tornava o projeto inteiro refém de uma etapa de alto risco — treinar um classificador de imagem com poucos dados próprios, sem garantia de prazo. Ao tratar a câmera como *"só mais uma fonte de eventos"*, o mesmo pipeline (`sessao_service.registrar_deteccao`) que hoje recebe cliques simulados e leituras de sensor de peso já está pronto para receber no construídas.

09 Privacidade e LGPD

O risco de privacidade identificado na Seção 02 já molda decisões de design implementadas, não apenas planejadas (fonte confirmada — comentários em `models.py`):

- Clientes são identificados por um **código de sessão temporário** (`SS-XXXXXX`), nunca por rosto ou identidade — decisão explicitamente registrada no código como princípio de design do projeto.
- Nenhuma tabela do modelo de dados atual armazena imagem, vídeo ou qualquer identificador biométrico de cliente.
- Para o Marco 3 (câmera real), o documento de investigação já recomenda: aviso visível de monitoramento, enquadramento restrito à zona de retirada do produto (nunca rosto/mão do cliente), e descarte da imagem após a inferência — pontos que continuam válidos e devem ser mantidos na implementação futura.

10 Próximos passos

`Marco 3`	Visão computacional real (`CAMERA_REAL`) — já especificada em documento próprio (`smartstock-vision-marco3.html/pdf`): coleta de dados, fine-tuning de YOLOv8/MobileNet, integração via novo endpoint.
`Depois`	Identificação de múltiplas unidades sobrepostas/ocultas — depende do modelo treinado no Marco 3.
`Depois`	Tela dedicada para o cliente (hoje só existe o painel interno do proprietário).
`Depois`	Deploy em produção (Render + banco na nuvem) — caminho já usado em outro projeto do grupo (FLUVIAL TRACK), reaproveitável aqui quando o grupo decidir publicar.

simulados e leituras de sensor de peso já está pronto para receber, no Marco 3, a saída de um modelo de visão computacional real — sem precisar reescrever a cesta virtual, o estoque ou o painel do proprietário.

06 O que já foi desenvolvido

O protótipo atual (fonte confirmada — código do repositório) já cobre um fluxo de ponta a ponta:

- **Cadastro de produtos** com preço, estoque, estoque mínimo e peso unitário (`models.py:30`).
- **Sessão de compra** identificada por um código único gerado (`SS-XXXXXX`), nunca por reconhecimento facial — decisão já implementada, não só planejada (`sessao_service.gerar_codigo_sessao`).
- **Detecção simulada** (clique manual, representando o que a câmera enviaria) e **detecção real por sensor de peso**: a variação de peso na prateleira é convertida em unidades inteiras e vira um evento automático, sem intervenção manual (`prateleira_service.registrar_leitura`).
- **Conferência de pagamento**: o sistema compara a quantidade detectada com a quantidade paga, ajusta o estoque pela quantidade *paga* (não pela detectada) e **nunca esconde uma divergência** — só reporta (`sessao_service.pagar_sessao`).
- **Histórico imutável de eventos** — cada retirada/devolução fica registrada permanentemente, com a fonte que a gerou (`EventoDeteccao`).
- **Painel do proprietário**: faturamento do dia, sessões em andamento, número de divergências e produtos abaixo do estoque mínimo (`routers/dashboard.py`).

Login de teste disponível (`proprietario@demo.smartstockvision.com`) para demonstração ao professor sem precisar cadastrar dados do zero.

11 Uso de IA na elaboração deste documento

Ferramenta: Claude Code, usada para (a) ler o documento de investigação técnica já produzido pelo grupo e o estado atual do repositório (modelos, serviços, README), e (b) reconstruir e redigir, a partir dessa leitura, a narrativa de "primeira ideia → desenvolvimento inicial" pedida pelo professor.

O que foi verificado diretamente no código antes de ser afirmado aqui: a existência e o comportamento do enum `FonteDeteccao` , a lógica de conferência de pagamento em `sessao_service.pagar_sessao` , a conversão de peso em unidades em `prateleira_service.registrar_leitura` , as tabelas e campos do banco de dados em `models.py` , e as métricas do painel em `routers/dashboard.py` — nada nas seções 06/07/08 é extrapolado sem correspondência direta no código.

O que é reconstrução, não transcrição literal: a Seção 01 (ideia original) foi inferida a partir do diagnóstico dela no documento de investigação — o texto exato da proposta original, se existiu como documento à parte, não estava disponível para esta redação. Isso está sinalizado explicitamente em vez de apresentado como citação direta.

A confirmar com o professor: os campos de disciplina/turma e integrantes do grupo, e se a narrativa de "dois pivots" corresponde de fato ao que foi discutido em sala — este documento organiza evidências já existentes (o PDF de investigação e o código), mas não substitui a memória do grupo sobre as conversas com o professor que não deixaram registro escrito.

12 Referências e documentos relacionados

Datasets e trabalhos citados na Seção 03 (lista completa, com DOI/arXiv, no documento de investigação técnica original):

- Goldman, E. et al. *Precise Detection in Densely Packed Scenes.* CVPR, 2019 (dataset SKU-110K).
- Wei, X.-S. et al. *RPC: A Large-Scale Retail Product Checkout Dataset.* arXiv:1901.07249, 2019.
- Jund, P. et al. *The Freiburg Groceries Dataset.* arXiv:1611.05799, 2016.
- Peng, J.; Xiao, C.; Li, Y. *RP2K: A Large-Scale Retail Product Dataset.* arXiv:2006.12634, 2020.
- Shoman, M. et al. *A Region-Based Deep Learning Approach to Automated Retail Checkout.* arXiv:2204.08584, 2022.
- BizTech Magazine. *IT Leaders Are Rethinking Retail Shrink With Computer Vision and Advanced Analytics.* 2025.
- BRASIL. Lei nº 13.709/2018 (LGPD). Presidência da República.

Documentos do próprio projeto usados como base para este relatório:

- *Investigação e Proposta Técnica — Visão Computacional para Inventário* (documento do grupo, Etapas 1 a 8).
- *SmartStock Vision — Marco 3* (especificação técnica da câmera real, já publicada).
- Repositório `smartstock-vision — README.md`, `backend/app/models/`, `backend/app/services/`, `backend/app/routers/dashboard.py`.

SmartStock Vision · Marco 1 — Ideação e desenvolvimento inicial · reconstruído a partir do documento de investigação técnica e do estado atual do repositório em 21/09/2026.